

АНАЛИЗ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Индивидуальное задание 8-го варианта: вычислить бесконечную сумму S с точностью ε .

Считать, что требуемая точность достигнута, если значение очередного слагаемого по модулю меньше заданного ε (это и все последующие слагаемые можно не учитывать). Определить и вывести количество слагаемых найденной суммы, учтенных при расчете. Значение суммы выводить с точностью до 8 знаков в дробной части. Если ни один из слагаемых не был учтен, то выдать об этом сообщение. Внимание! Для нахождения степени числа не использовать стандартных функций, а вычислять его самостоятельно с помощью оператора цикла. Для вычисления факториала числа применить цикл for.

Ограничения: x, ε – действительные числа ($x \neq 0, \varepsilon > 0$), a – целое число ($|a| < 106$)

Данные в программу вводит пользователь с клавиатуры, а правильность вводимых значений не гарантируется. Поэтому перед выполнением вычислений следует проверить валидность входных данных. В случае если входные данные не валидны пользователю необходимо ввести их заново.

$$S = b + \sum_{k=1}^{\infty} \left(\frac{(|a - k| + 1)!}{k! * 2k} * x^k \right)$$
$$b = \begin{cases} 1/a, & \varepsilon < 1 \\ a!, & \varepsilon \geq 1 \end{cases}$$

ВЫПОЛНЕНИЕ ЗАДАНИЯ

```
#include <stdio.h>
#include <conio.h>
#include <iostream>
#include <iomanip>
#include <Windows.h>
using namespace std;

// Вычисление факториала через цикл for
int factorial(int a)
{
    if (a != 0)
    {
        int b = 1;
        for (int i = 1; i <= a; i++)
            b *= i;
        return b;
    }
    return 1;
}

// Перегрузка вычисления факториала через цикл for для
действительных чисел
double factorial(double a)
{
    if (a != 0)
    {
        double b = 1;
        for (int i = 1; i <= a; i++)
            b *= i;
        return b;
    }
    return 1;
}

// Функция возведения в степень
int pow(int& a, int& n)
{
    for (int i = 0; i < n; i++)
        a *= a;
    return a;
}
```

```

// Перегрузка функции возведения в степень для действительных чисел
double pow(double a, int n)
{
    for (int i = 0; i < n; i++)
        a *= a;
    return a;
}

int main()
{
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    setlocale(LC_ALL, "Rus");

    cout << fixed;
    cout.precision(8);
    int a = 2;
    double x = 1, e = 2;
    double b = e < 1 ? 1 / a : factorial(a);

    double S = 0;
    double Sk = 0;

    int menu;
    cout << "Выберите пункт меню\n";
    cout << "1. Задать значения переменных a,x,e вручную\n";
    cout << "2. Использовать значения переменных по умолчанию {a
= 2, x = 1, e = 2}\n";
    cin >> menu;
    switch (menu)
    {
    case 1:
        do
        {
            cout << "[a -- целое, |a| < 10^6] a = ";
            cin >> a;
            cout << "[x -- действительное, x != 0] x = ";
            cin >> x;
            cout << "[e -- действительное, e > 0] e = ";
            cin >> e;
            cout << "a = " << a << "; x = " << x << "; e = " << e <<
endl;
        } while (a > pow(10, 6) || x == 0 || e < 0);
    }
}

```

```

default:
    break;
}

for (int k = 1; Sk < e; k++) // Подсчет суммы
{
    double delimoe = factorial(abs(a - k) + 1) * pow(x, k);
    double delitel = (factorial(k) * 2 * k);
    if (delitel != 0)
        Sk += delimoe / delitel;
    else
    {
        cout << "Деление на ноль, выход из цикла\n";
        break;
    }
    cout << "S[" << k << "]:\t" << Sk << endl;
}
if (Sk == 0)
    cout << "Ни одно из слагаемых не было учтено\n";
S = b + Sk;
cout << "S = \t" << S << endl;

return 0;
}

```

Результат выполнения программы (снимок экрана):

The screenshot shows the Visual Studio IDE with a C++ file named 'OAIp LR 1.cpp'. The code implements a loop that calculates terms of a series and adds them to a sum S until the sum exceeds a given value e. The console output shows the following results:

```

Выберите пункт меню
1. Задать значения переменных a,x,e вручную
2. Использовать значения переменных по умолчанию {a = 2, x = 1, e = 2}
2
S[1]: 1.00000000
S[2]: 1.12500000
S[3]: 1.18055556
S[4]: 1.21180556
S[5]: 1.23180556
S[6]: 1.24569444
S[7]: 1.25589853
S[8]: 1.26371103
S[9]: 1.26988387
S[10]: 1.27488387
S[11]: 1.27901610
S[12]: 1.25027548
S[13]: 0.88356627
S[14]: 2.21534696
S = 2.21534696
C:\Users\minsk\source\repos\OAIp LR 1\Debug\OAIp LR 1.exe (процесс 10060) завершил работу с кодом 0.
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" -> "Параметры" -> "Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно.

```

Блок-схема алгоритма изображена на рисунке 2.1

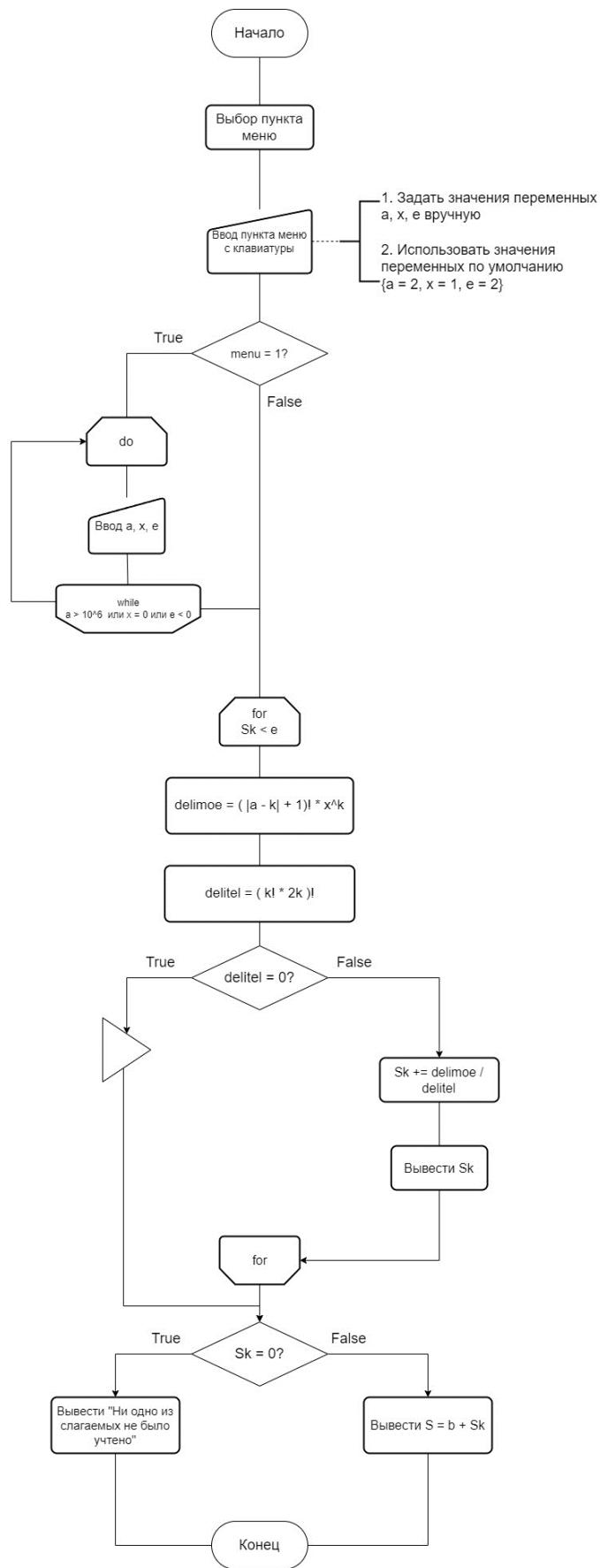


Рисунок 2.1 – Блок-схема алгоритма

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Понятия «решение», «проект» и «конфигурация» в MS Visual Studio. Решение – контейнер связанных между собой проектов.

Проект – совокупность всех модулей и файлов различных типов программы.

Конфигурация – тип сборки, описанный с именованным набором свойств, обычно параметров компилятора и расположений файлов.

2. Возможности рабочего пространства проекта.

Цвета и шрифты. Вы можете настраивать цвета и шрифты среды Visual Studio в окне средства→параметры→окружение→шрифты и цвета.

Перед этим лучше выбрать цветовую схему в окне средства→параметры→окружение→цветовая схема, после чего можно «уточнять» цветовую схему дополнительными настройками шрифтов и цвета.

Горячие кнопки. Вы можете настроить горячую кнопку (сочетание клавиш) на любую команду в Visual Studio. Для этого откройте окно средства→параметры→окружение→клавиатура, введите название команды и сочетание клавиш для вызова этой команды.

Плагины. Настоятельно рекомендую установить плагин JetBrains ReSharper, который внедряется в Visual Studio и предоставляет много полезных инструментов для работы с кодом.

Зайти внутрь метода. Вы можете зайти внутрь вызываемого метода и посмотреть его код. Для этого нужно нажать на Ctrl и кликнуть мышкой по методу, либо установить каретку на методе и нажать F12.

Команда работает не только для методов, но и вообще для всего (классов, переменных и. т. д.). ReSharper интегрирует эту команду с дизассемблером, благодаря этому можно смотреть код метода, даже если в проекте нет его исходников.

Выйти наружу метода. Вы можете найти все места в коде, где используется выбранный метод. Для этого кликните по методу мышкой и выберите в контекстном меню «Найти все ссылки». У ReSharper есть аналогичная команда, но более продвинутого (контекстное меню→find usages). Помимо отображения всех мест где используется этот метод, ReSharper может сразу же перейти в то место, откуда вызывается указанный метод, что очень удобно. Рекомендуется настроить горячую кнопку на эту команду.

Поиск метода. Помимо стандартного поиска Ctrl+F, в студии есть и более удобный поиск с подсказками. Запустите команду правка→перейти→перейти к символу и начните набирать название метода. Студия будет сама подсказывать варианты, к какому методу перейти. Похожий инструмент есть и в ReSharper.

(resharper→navigate→goto symbol). Рекомендуется настроить удобную горячую кнопку на эту команду.

Переименование. Вы можете переименовывать классы, методы и переменные с помощью команды студии контекстное меню→переименовать. Решарпер поддерживает и более продвинутое переименование (resharper→refactor→rename либо F2). В частности, решарпер при переименовании класса может заодно переименовать файл с этим классом плюс экземпляры этого класса.

Выделение фрагмента кода в метод. Вы можете перенести выделенный фрагмент кода в отдельный метод, запустив команду правка→рефакторинг→извлечь метод.

3. Этапы построения консольного приложения.

1. Создание решения
2. Добавление проекта
3. Настройка конфигурации проекта
4. Создание файлов и модулей

4. Понятия «препроцессор» и «условная компиляция» в C++.

Препроцессор — это директива, которые дают инструкции компилятору для предварительной обработки информации до начала фактической компиляции.

Директивы условной компиляции позволяют в зависимости от условий добавить в файл определенный код.

5. Назначение команд препроцессора #include, #define, #undef, #if, #ifdef, #else, #elif, #error, #pragma.

#include — используется для подключения системных файлов.

#define — определяет идентификатор и последовательность символов, которой будет за-мещаться данный идентификатор при его обнаружении в тексте программы.

#undef — удаляет имя, ранее созданное с помощью #define , то есть отменяет его определение.

#if — Если написанное выражение (после #if) имеет ненулевое значение, группа строк сразу после директивы #if хранится в модуле перевода.

#ifdef — Если идентификатор не определен или его определение было удалено с #undef , условие имеет значение true (nonzero). В противном случае условие не выполняется (false, значение равно 0).

#if, #ifdef и #ifndef, оказывается истинным, то будет скомпилирован код, расположенный между одной из этих трех директив и директивой #endif; в противном случае данный код будет опущен. Директива #endif используется для обозначения конца блока #if. Директиву #else можно

использовать с любой из перечисленных выше директив для предоставления альтернативного варианта компиляции.

#error — Директива **#error** выдает сообщение об ошибке, указанное пользователем во время компиляции, а затем завершает компиляцию.

#pragma — это директива, используемая для указания определенных настроек и инструкций для компилятора или интерпретатора.

6. Понятия «оператор», «операнд», «идентификатор», «переменная» и «константа» в C++.

Оператор — это элемент языка программирования, который задает действие над данными или управляет порядком выполнения программы.

Операндами традиционных арифметических операций (+ - * /) могут быть константы, переменные, элементы массивов, любые арифметические выражения.

Идентификатор — имя, задаваемое в программе для переменных, типов, функций и меток.

Переменная — именованный объект, который хранит данные определенного типа и может изменять их в процессе выполнения программы.

Константа — именованный или безымянный объект, который хранит данные определенного типа и не может изменять их в процессе выполнения программы.

7. Приоритетность операций и порядок обработки выражений в C++.

Приоритет	Операция	Ассоциативность	Описание
1	::	слева направо	унарная операция разрешения области действия
	[]		операция индексирования
	()		круглые скобки
	.		обращение к члену структуры или класса
	->		обращение к члену структуры или класса через указатель
2	++	слева направо	постфиксный инкремент
	--		постфиксный декремент

Приоритет	Операция	Ассоциативность	Описание
3	++	справа налево	префиксный инкремент
	—		префиксный декремент
4	*	слева направо	умножение
	/		деление
	%		остаток от деления
5	+	слева направо	сложение
	—		вычитание
6	>>	слева направо	сдвиг вправо
	<<		сдвиг влево
7	<	слева направо	меньше
	<=		меньше либо равно
	>		больше
	>=		больше либо равно
8	==	слева направо	равно
	!=		не равно
9	&&	слева направо	логическое И
10		слева направо	логическое ИЛИ
11	?:	справа налево	условная операция (тернарная операция)
12	=	справа налево	присваивание
	*=		умножение с присваиванием
	/=		деление с присваиванием
	%=		остаток от деления с присваиванием

Приоритет	Операция	Ассоциативность	Описание
	+=		сложение с присваиванием
	-=		вычитание с присваиванием
13	,	слева направо	Запятая

8. Понятие типизации. Правила преобразования значений операндов из одного типа в другой в C++.

Типизация — совокупность правил в языках программирования, назначающих свойства, именуемые типами, различным конструкциям, составляющим программу — переменные, выражения, функции и модули.

C++ позволяет нам преобразовывать данные одного типа в данные другого — это известно как преобразование типов.

Преобразование типа, которое автоматически выполняется компилятором, известно как неявное преобразование.

Те преобразования, при которых не происходит потеря информации, являются безопасными. Как правило, это преобразования от типа с меньшей разрядностью к типу с большей разрядностью. В частности, это следующие цепочки преобразований:

bool -> char -> short -> int -> double -> long double

bool -> char -> short -> int -> long -> long long

unsigned char -> unsigned short -> unsigned int -> unsigned long

float -> double -> long double

9. Порядок автоматического приведения типов в выражениях в C++.

Приведение типов целых чисел.

Если в выражении используются операторы разных целочисленных типов (например, int и long), то происходит автоматическое приведение к более широкому типу. Например, если у нас есть выражение intNum + longNum, то intNum будет автоматически преобразован в тип long перед выполнением операции.

Приведение типов с плавающей точкой.

Если в выражении используются операторы разных типов с плавающей точкой (например, float и double), то происходит автоматическое приведение к более точному типу. Например, если у нас есть выражение floatNum + doubleNum, то floatNum будет автоматически преобразован в тип double перед выполнением операции.

Приведение типов целых чисел и с плавающей точкой.

Если в выражении используются операторы целых чисел и операторы с плавающей точкой, то операнды с целочисленным типом будут автоматически преобразованы в тип с плавающей точкой перед выполнением операции. Например, выражение `intNum + floatNum` будет работать, но со значениями типа `intNum`.

Приведение типов перечислений.

Если в выражении используются операторы перечислений, то автоматического приведения типов не происходит. Если же нужно привести перечисление к определенному типу данных, можно использовать статическое приведение типов с помощью оператора `static_cast`.

Приведение типов указателей.

Если в выражении используются указатели на разные типы, то автоматического приведения типов не происходит. Приведение указателей может быть выполнено с помощью операторов `static_cast`, `const_cast`, `reinterpret_cast` или `dynamic_cast`.

Автоматическое приведение типов может привести к потере точности или неожиданным результатам.

10. Средства и особенности языка C++ при организации разветвляющегося процесса.

`if-else` — позволяет выполнять определенные блоки кода в зависимости от условия.

`switch` — позволяет выбирать один из нескольких блоков кода для выполнения, в зависимости от значения выражения.

11. Возможности языка C++ для организации циклического процесса.

`for` — позволяет выполнить блок кода определенное количество раз, основываясь на заданных условиях.

`while` — позволяет выполнять блок кода, пока указанное условие истинно.

`do-while` — позволяет выполнить блок кода, а затем проверить условие для продолжения выполнения.

12. Особенности организации в C++ итеративного процесса. Цикл `for`. Синтаксис цикла `for` выглядит следующим образом:

```
for (инициализация; условие; обновление) {  
    // блок кода  
}
```

Здесь:

- 'инициализация' - это выражение, выполняемое перед началом цикла. Оно инициализирует переменные, которые будут использоваться в цикле. Например, можно создать и инициализировать переменную счетчика, которая будет отслеживать количество повторений цикла.
- 'условие' - это выражение, которое проверяется перед каждой итерацией цикла. Если оно истинно, цикл продолжает выполняться. Если оно ложно, цикл завершается.
- 'обновление' - это выражение, которое выполняется после каждой итерации цикла. Обычно используется для обновления переменных.

13. Инструментальная панель отладчика в среде Visual C++.

Отладка приложения означает запуск и выполнение приложения с подключенным отладчиком. При этом в отладчике доступно множество способов наблюдения за выполнением кода. Вы можете пошагово перемещаться по коду и просматривать значения, хранящиеся в переменных, задавать контрольные значения для переменных, чтобы отслеживать изменение значений, изучать путь выполнения кода, просматривать выполнение ветви кода и т. д.

Инструментальная панель отладчика позволяет:

1. Останавливать выполнение кода в точках останова.
2. Использовать команды для пошагового выполнения кода в отладчике.
3. Проверять переменные в подсказках к данным и окнах отладчика.
4. Просматривать стек вызовов.

14. Точки останова и точки трассировки. Настройки отладчика и пошаговое выполнение программы.

Точка трассировки (tracerepoint) — это точка останова, которая выводит сообщение в окно вывода. Точка трассировки может играть роль временного оператора трассировки в языке программирования и не приостанавливает выполнение кода. Вы создаете точку трассировки, задавая особое действие в окне Параметры точки останова.

Точка останова — это преднамеренное прерывание выполнения программы, при котором выполняется вызов отладчика (одновременно с этим программа сама может использовать точки останова для своих нужд).

15. Возможности языка C++ по организации ввода и вывода данных для консольного приложения. Форматированный вывод данных.

Вывод

```
std::cout << "Name: " << "Tom" << "\n";
printf("1. просто строка \n");
printf("2. целое число: %d \n", 1234);
printf("3. дробное число: %f \n", 12.34);
```

```
printf("4. Символ: %c \n", 'C');  
printf("5. строка: %s \n", "C++");  
printf("6. процент: 100%% \n");
```

Ввод

```
std::cin >> age;  
printf("Введите дату, название месяца и цену (например, 15 Август 2023  
$10.50): ");  
int n = scanf("%d %s %*d $%f", &day, month, &price);
```

Форматированный вывод данных в C++ может быть осуществлен с помощью манипуляторов. Манипуляторы - это специальные объекты, которые изменяют параметры вывода. Некоторые из них:

- `std::setw(n)` - устанавливает ширину поля вывода в n символов.
- `std::setprecision(p)` - устанавливает точность вывода для чисел с плавающей запятой в p знаков после запятой.
- `std::setfill(c)` - устанавливает символ c для заполнения пустых мест в поле вывода.
- `std::left` и `std::right` - выравнивание вывода по левому и правому краю соответственно.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены теоретические сведения, составлена блок-схема индивидуального задания и реализовано программное средство, удовлетворяющее поставленной задаче, даны ответы на контрольные вопросы.